

Trabajo Práctico:

Estimación y Reducción de Potencia en un Interpolador FIR

Inserción de ceros contra descomposición polifásica

Diseño de Circuitos Integrados VLSI

Flujo Synopsys SAED14: VCS · Fusion Compiler · PrimeTime PX

Fecha de entrega: 26 de octubre de 2026

Índice

1. Objetivos	3
2. Repaso teórico	3
2.1. Interpolación e inserción de ceros	3
2.2. Descomposición polifásica	4
2.3. Componentes de la potencia	4
3. Entorno real de trabajo	5
3.1. Herramientas	5
3.2. Librería	5
3.3. Template de flujo	5
4. Diseños	5
4.1. Interfaz común	6
5. Condiciones comunes	7
6. Ejercicios	7
Ejercicio 1: Implementación, verificación y cierre de timing	7
Ejercicio 2: Anatomía del consumo	8
Ejercicio 3: Glitches	9
Ejercicio 4: Data gating	10
Ejercicio 5: Clock gating	10
Ejercicio 6: Leakage: multi-V_t	11
Ejercicio 7: (Opcional) MAC compartido y simetría	12
7. Ayudas y recordatorios	12
8. Entregables	13
8.1. Tabla 1: comparativa de arquitecturas (la principal)	13
8.2. Tabla 2: variantes con técnicas de reducción	14
8.3. Informe	14

8.4. Cronograma sugerido (4 semanas)	14
8.5. Criterios de evaluación	15
A. Plantillas de scripts	15
A.1. Testbench: dump de FSDB	15
A.2. Fusion Compiler: síntesis	16
A.3. PrimeTime PX: potencia	17
A.4. Actividad supuesta (sin simulación)	17

1. Objetivos

En este TP van a **escribir dos implementaciones RTL del mismo interpolador** y comparar cuánta potencia consume cada una, y sobre todo **por qué**.

La primera es la versión directa del libro: se sube la tasa insertando $L - 1$ ceros entre muestras y se filtra con un FIR común que corre a la tasa alta. Es correcta, pero hace un trabajo enorme al pedo: $L - 1$ **de cada L multiplicaciones son por cero**. El multiplicador conmuta, gasta potencia, y el producto que entrega es cero.

La segunda es el **interpolador polifásico**: la misma respuesta en frecuencia, pero reordenado de manera tal que esas multiplicaciones por cero **directamente no existen**. Cada subfiltro trabaja a la tasa baja y un conmutador arma la salida.

Las dos producen la **misma secuencia de salida, bit a bit**. Solo cambia cómo está organizado el hardware. Todo el TP consiste en medir qué cuesta esa diferencia, en watts, y entender de dónde sale.

Al terminar, cada grupo debe ser capaz de:

1. **Implementar y verificar** las dos arquitecturas, y demostrar que son funcionalmente equivalentes.
2. Separar y justificar las componentes de potencia: *switching*, *internal* y *leakage*.
3. Explicar por qué la estimación depende del estímulo, y por qué una actividad obtenida de GLS con SDF es más creíble que una supuesta.
4. Identificar qué parte del diseño domina el consumo (red de reloj, registros, multiplicadores, sumadores) y atacar esa parte.
5. Medir la potencia de **glitch** y explicar, con los retardos del SDF en la mano, por qué una arquitectura glitchea más que la otra.
6. Aplicar **data gating**, *clock gating* y multi- V_t , y cuantificar cuánto aporta cada uno.
7. Reportar todo en términos comparables: potencia, energía por muestra, área y slack.

Nota

Este TP asume lo visto en la clase de GLS, SDF, glitches y potencia. Las secciones 3 (entorno) y 7 (ayudas y recordatorios) conviene leerlas **antes** de empezar: ahí está resumido lo que hay que tener en cuenta al correr el flujo.

2. Repaso teórico

2.1. Interpolación e inserción de ceros

Para subir la tasa de muestreo por un factor entero L se intercalan $L - 1$ ceros entre muestras consecutivas:

$$x_u[n] = \begin{cases} x[n/L] & \text{si } n \text{ es múltiplo de } L, \\ 0 & \text{en otro caso,} \end{cases} \quad (1)$$

y se filtra con un pasabajos $h[k]$ de N taps que elimina las imágenes espectrales:

$$y[n] = \sum_{k=0}^{N-1} h[k] x_u[n - k]. \quad (2)$$

El filtro corre a la tasa alta Lf_s y hace N multiplicaciones por cada muestra de salida. Pero la línea de retardo está llena de ceros: en cualquier instante, solo N/L de los N valores son muestras reales. **El resto de los productos da cero.**

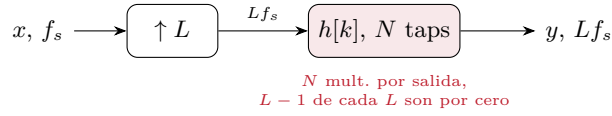
2.2. Descomposición polifásica

Escribiendo el índice de salida como $n = mL + p$, con $p = 0, 1, \dots, L-1$ la fase, el término $x_u[n-k]$ es distinto de cero solo cuando $k \equiv p \pmod{L}$. Reemplazando $k = rL + p$ en la ecuación (2):

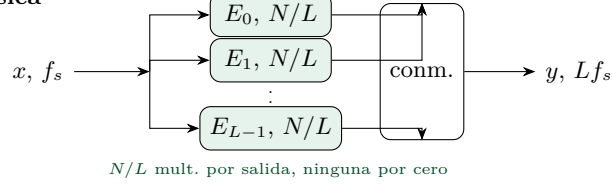
$$y[mL + p] = \sum_{r=0}^{N/L-1} \underbrace{h[rL + p]}_{E_p[r]} x[m - r]. \quad (3)$$

Es decir: **la salida de fase p es la convolución de la entrada original con el subfiltro E_p , de N/L taps, que corre a la tasa baja f_s . Las L fases se calculan en paralelo y un conmutador las entrega una por ciclo.**

Ingenua



Polifásica



Ahorro teórico. Multiplicaciones por segundo:

$$R_{\text{ingenua}} = N \cdot Lf_s, \quad R_{\text{poli}} = \frac{N}{L} \cdot Lf_s = Nf_s \quad \Rightarrow \quad \frac{R_{\text{ingenua}}}{R_{\text{poli}}} = L. \quad (4)$$

Con $L = 4$, la versión polifásica hace **la cuarta parte** del trabajo aritmético. Cuánto de eso se traduce en ahorro de potencia es justamente lo que hay que medir: hay reloj, registros y control que no escalan igual.

2.3. Componentes de la potencia

La potencia total de un circuito CMOS se descompone como

$$P_{\text{total}} = \underbrace{P_{\text{switching}} + P_{\text{internal}}}_{P_{\text{dinámica}}} + P_{\text{leakage}}. \quad (5)$$

Conmutación (switching). Energía para cargar y descargar la capacidad de las *nets* (cableado más pines de entrada de las celdas que manejan):

$$P_{\text{sw}} = \frac{1}{2} \sum_{i \in \text{nets}} C_i V_{DD}^2 TR_i = \alpha C_L V_{DD}^2 f_{\text{clk}}, \quad (6)$$

con TR_i la tasa de transiciones de la net i y α el factor de actividad respecto del reloj. Depende de los datos, de la frecuencia, de V_{DD}^2 y de la capacidad.

Interna (internal). Se disipa *dentro* de la celda: nodos internos y corriente de corto-circuito durante la transición (NMOS y PMOS conduciendo a la vez). Sale de tablas de la librería indexadas por *slew* de entrada y carga de salida: transiciones lentas aumentan P_{int} . En los registros se consume en cada flanco de reloj **aunque el dato no cambie**.

Glitches. Transiciones espurias por caminos de distinto retardo que convergen en una compuerta (típico en árboles de sumadores y multiplicadores). Requieren cuatro condiciones: *convergencia* de señales que conmutan, *desbalance* de tiempos de llegada, *sensibilización* de la compuerta y que el pulso *sobreviva* al filtrado inercial. Solo se observan en simulación con retardos (SDF).

Fugas (leakage). Consumo estático, presente sin actividad:

$$P_{\text{leak}} = V_{DD} \cdot (I_{\text{sub}} + I_{\text{gate}} + I_{\text{junction}}), \quad I_{\text{sub}} \propto e^{\frac{V_{GS} - V_t}{n V_T}}. \quad (7)$$

Depende exponencialmente de V_t y de la temperatura, depende del estado lógico de las entradas y es proporcional al área. **No depende de la frecuencia.**

Métrica de comparación. Como las variantes cambian el trabajo por muestra, comparar además la energía por muestra de salida:

$$E_{\text{muestra}} = \frac{P_{\text{total}}}{L f_s} \quad [\text{pJ/muestra}]. \quad (8)$$

3. Entorno real de trabajo

3.1. Herramientas

Etapas	Herramienta	Comando
Simulación	VCS	vcs
Ondas	Verdi	verdi, fsdb2saif
Síntesis y P&R	Fusion Compiler	fc_shell (compile_fusion)
Potencia	PrimeTime PX	pt_shell
Lint	SpyGlass	sg_shell

3.2. Librería

Librería educativa **SAED14**, en `libs_14/`:

- Cuatro *flavors*: `hvt`, `rvt`, `lvt`, `slvt`, cada uno con `base` y `cg` (clock gating).
- Corners `.db` disponibles: `ss0p72v125c`, `ss0p72vm40c`, `tt0p8v25c`, `tt0p8v125c`, `tt0p8vm40c`, `ff0p88vm40c`.
- `.ndm` (físico más timing) en `libs_14/ndm/`, tech file y TLU+ en `libs_14/physical/`.
- Modelos Verilog para GLS en `/eda/tools/pdk/SAED14_PDK/EDK/SAED14nm_EDK_STD_*/verilog/`

Peor caso de setup: `ss0p72v125c` (lento, baja tensión, caliente). **Peor caso de hold:** `ff0p88vm40c`.

3.3. Template de flujo

Usar el template genérico del curso. Copiar la estructura de un módulo existente y cambiar **una sola línea** del Makefile (`TOP = ...`). Los scripts leen el diseño de variables de entorno, así que no hay que tocarlos.

4. Diseños

El RTL lo escriben ustedes en este TP. Las dos arquitecturas implementan el mismo interpolador y deben entregar la misma secuencia de salida.

Target	Qué hace
make run	Compila y simula el RTL; genera <code>sim/waves/<top>_rtl.fsdb</code>
make syn	Síntesis; genera netlist, SDC y SDF en <code>syn/outputs/</code>
make gls	GLS post-síntesis con SDF anotado y chequeos de timing
make gls-ntc	Igual pero con <code>+notimingcheck</code> (delays sí, chequeos no)
make saif	Convierte el FSDB de GLS a SAIF
make power	PrimeTime PX con la actividad del SAIF
make pnr	Floorplan, place, CTS y ruteo; genera netlist, SDF y SPEF
make gls-pnr	GLS post-P&R
make waves, waves-gls	Abren Verdi

Diseño	Descripción
interp_zeros	Versión ingenua: inserción de $L - 1$ ceros más un FIR de N taps corriendo a la tasa alta. La línea de retardo contiene los ceros intercalados y los multiplicadores los procesan como cualquier otro dato.
interp_poly	Versión polifásica: L subfiltros E_p de N/L taps que trabajan sobre la entrada original a f_s , más un conmutador que entrega una fase por ciclo. Mismos coeficientes, repartidos según $E_p[r] = h[rL + p]$.

Parámetros. $L = 4$, $N = 16$ taps (o sea 4 subfiltros de 4 taps), datos y coeficientes de 12 bits en Q1.11, acumulador con bits de guarda, salida redondeada a 12 bits. Coeficientes de un pasabajos de fase lineal que cumpla la función de anti-imagen.

Nota

$N = 16$ y **12 bits**. Con 32 taps de 16 bits la síntesis de cada variante pasa de minutos a bastante más, y hay que correr el flujo completo unas 10 veces. El fenómeno que se quiere observar se ve igual.

Nota

Las dos versiones deben ser bit a bit equivalentes. Mismos coeficientes, mismo esquema de redondeo y misma ganancia (si escalan por L para compensar la pérdida de amplitud de la interpolación, escalar en las dos). Si difieren, cualquier diferencia de potencia que midan puede venir de ahí y el TP pierde sentido. La latencia sí puede diferir: alinear antes de comparar.

4.1. Interfaz común

Las dos variantes exponen la misma interfaz, para poder reusar el testbench y el flujo sin cambios. **El reloj es único y corre a la tasa alta** $f_{\text{clk}} = Lf_s$: entra una muestra nueva cada L ciclos (`i_valid` alto 1 de cada L) y sale una muestra por ciclo.

```
module interp_zeros #(
    parameter int L = 4, // factor de interpolacion
    parameter int N = 16, // taps del prototipo h[k]
    parameter int DW = 12, // ancho de dato
    parameter int CW = 12 // ancho de coeficiente
)()
    input logic i_clk,
    input logic i_rst,
    input logic i_valid, // alto 1 de cada L ciclos
    input logic signed [DW-1:0] i_data, // muestra a f_s
    output logic o_valid, // alto todos los ciclos
    output logic signed [DW-1:0] o_data // salida a L*f_s
```

```
);
```

Los coeficientes van como `localparam` dentro del módulo. La salida debe estar **registrada** y `o_valid` acompañar al dato válido.

5. Condiciones comunes

Salvo indicación en contrario:

- **Período de reloj:** el que determinen en el Ejercicio 1. El mismo para todas las variantes que se comparen entre sí.
- **Corner de síntesis:** `ss0p72v125c` (el del template).
- **Estímulo estándar:** 2000 muestras de entrada (a f_s) de una suma de dos tonos en banda de paso más ruido blanco de baja amplitud, a -6 dBFS. Generarlo una vez con el modelo de referencia, guardarlo en un archivo y **usar el mismo en todas las mediciones**.
- **Toda medición de potencia** se hace con SAIF obtenido de **GLS con SDF**, salvo donde el ejercicio pida otra cosa.
- **Verificar siempre la anotación:** si `report_switching_activity -list_not_annotated` no da prácticamente 100 % de nets anotadas, el número de potencia no vale.
- Toda variante debe **volver a pasar la verificación funcional**: misma salida bit a bit que el modelo de referencia.
- Guardar todos los reportes en `reports/ejN/`.

6. Ejercicios

Ejercicio 1: Implementación, verificación y cierre de timing *Las dos arquitecturas*

Objetivo

Escribir las dos arquitecturas, demostrar que son equivalentes, y dejar el flujo andando de punta a punta. Sin esto, los demás ejercicios no se pueden comparar.

- (a) **Modelo de referencia en Python:** diseñar el pasabajos de $N = 16$ taps, cuantizarlo a Q1.11 y escribir el interpolador con la misma aritmética de punto fijo que va a tener el RTL (mismos bits de guarda, mismo redondeo). Verificar en el modelo que la descomposición polifásica de la ecuación (3) da **exactamente** lo mismo que la inserción de ceros de la ecuación (2). Este modelo es la verdad contra la que se comparan las dos implementaciones.
- (b) **Implementar `interp_zeros`** y verificarla en RTL con el estímulo estándar (`make run`): salida **bit a bit** igual al modelo.
- (c) **Implementar `interp_poly`** y verificarla contra el mismo modelo. Documentar la latencia de cada una y cómo las alinearon.
- (d) **Contar multiplicadores:** reportar cuántos multiplicadores instancia cada versión y cuántas multiplicaciones efectivas hace por muestra de salida. Contrastar con el ahorro teórico L .

- (e) **Buscar el período mínimo de cada una:** sintetizar con períodos decrecientes (por ejemplo 3 ns, 2 ns, 1,5 ns, 1,2 ns y 1,0 ns) y anotar el WNS. Reportar el $f_{\text{máx}}$ de cada arquitectura y explicar la diferencia mirando el camino crítico.
- (f) Fijar como **período de trabajo común** el mínimo que cierren **las dos**, más un 20 % de margen. Justificar: con slack muy grande la herramienta usa todo HVT y el multi- V_t del Ejercicio 6 no muestra nada; con slack cero cualquier cambio rompe el timing y no se puede comparar.
- (g) Correr `make gls` sobre las dos y analizar las violaciones que aparezcan (ver Sección 7). Después `make gls-ntc` y verificar que la salida de la GLS coincide con la del RTL.
- (h) Correr `make saif` y `make power` en las dos. Reportar las tres componentes y el porcentaje de nets anotadas.

Preguntas para el informe

- ¿Cuál es el camino crítico de cada arquitectura? ¿Por qué la polifásica suele cerrar a mayor frecuencia?
- ¿Qué violaciones aparecieron en (g) y cuáles eran reales?
- ¿La relación de potencias se parece al factor L del ahorro teórico? Si no, ¿qué parte del circuito no escala?

Ejercicio 2: Anatomía del consumo

Switching, internal y leakage

Objetivo

Descomponer la potencia de las dos arquitecturas y verificar experimentalmente de qué depende cada componente.

- (a) Con el reporte de PrimeTime PX, construir para **cada arquitectura** un gráfico de torta de la potencia por grupo (*clock_network*, *register*, *combinational*) y otro por componente (switching, internal, leakage). Ponerlos lado a lado. Usar `report_power -hierarchy` para ver qué instancia consume más en cada una.
- (b) **Aislar los multiplicadores:** del reporte jerárquico, extraer qué fracción de la potencia total se va en los multiplicadores de cada versión. Esta es la comparación más directa del efecto de multiplicar por cero.
- (c) **Barrido en frecuencia:** repetir el análisis a la mitad y al doble del período de trabajo, con el mismo netlist y el mismo SAIF. Graficar las tres componentes contra f .
- (d) **Barrido de corners:** analizar el mismo netlist en `tt0p8v25c`, `tt0p8v125c` y `ss0p72v125c`. Tabular las tres componentes.
- (e) **Reloj sin datos:** simular con `i_valid` en cero y la entrada quieta, con el reloj corriendo. Medir cuánta potencia queda en cada arquitectura.

Preguntas para el informe

- Verificar que P_{sw} y P_{int} escalan linealmente con f y que P_{leak} es constante. ¿Sus gráficos lo cumplen?
- ¿Cuánto crece el leakage entre 25 °C y 125 °C? ¿Qué implica para un equipo que trabaja caliente?
- En (e), ¿qué fracción de la potencia dinámica consumen reloj y registros sin procesar información útil? Esto motiva el Ejercicio 5.
- En la versión ingenua, ¿el multiplicador que recibe un cero consume realmente menos que uno que recibe una muestra? Mirar la actividad de sus nets internas en el SAIF y justificar.

Ejercicio 3: Glitches*Dinámica de glitch***Objetivo**

Medir la potencia debida a glitches en las dos arquitecturas y explicar la diferencia con los retardos en la mano.

- (a) Para **interp_zeros**, generar **dos** SAIF de la simulación de compuertas: uno con la GLS **funcional** (delay cero, **+define+FUNCTIONAL**) y otro con la GLS **con SDF**. Analizar ambos en PrimeTime PX. La diferencia de potencia dinámica es la estimación de la potencia de glitch.
- (b) Contar los glitches directamente: en el SAIF, el campo IG de cada net. Identificar las 10 nets con más glitches y ubicarlas en el netlist. ¿Están donde esperaban?
- (c) Repetir (a) y (b) para **interp_poly**. Armar una tabla comparativa: potencia de glitch, potencia total, glitches totales (suma de IG), área, cantidad de registros y slack.
- (d) Tomar la net con más glitches de la versión ingenua y **rastrear en el SDF** los tiempos de llegada de las señales que convergen en ella. Mostrar el Δt que produce el glitch. Hacer lo mismo con la net equivalente de la polifásica y comparar los Δt .
- (e) Abrir en Verdi el FSDB de la GLS con SDF y **mostrar una captura** de una net donde se vea el glitch.

Preguntas para el informe

- ¿Cuál de las dos glitchea más y por qué? Relacionar con las cuatro condiciones del repaso teórico y con el tamaño del árbol de sumadores de cada una.
- Las transiciones muestra $\rightarrow$ cero $\rightarrow$ muestra de la versión ingenua, ¿ayudan o empeoran la actividad de glitch?
- ¿Por qué un SAIF obtenido de simulación RTL nunca puede mostrar glitches?
- En clase medimos IG 0 en el PRBS aun después de P&R. ¿Qué tiene este filtro que no tenía el PRBS?

Ejercicio 4: Data gating

Switching en lógica combinacional

Objetivo

Recuperar parte del ahorro del polifásico **sin cambiar la arquitectura**, evitando que conmute la lógica cuyo resultado se sabe de antemano.

En `interp_zeros` hay $L - 1$ de cada L multiplicadores cuyo operando de datos es cero: el producto va a dar cero, pero las entradas del multiplicador igual conmutan y su lógica interna también. El **data gating** consiste en congelar esas entradas para que no propaguen actividad.

- Data gating manual:** con el contador de fase se sabe qué taps tienen cero. Implementar el aislamiento de dos formas y medir las dos:
 - **forzando a cero** el operando (Listado 1);
 - **reteniendo** el último valor con un registro o latch, y forzando el producto a cero a la entrada del sumador.
- Comparar contra la opción automática de la herramienta (`set_operand_isolation_style`) y ver si infiere lo mismo.
- Evaluar el impacto en timing: la compuerta de aislamiento queda en el camino de datos. Reportar el slack antes y después.
- Cerrar el círculo:** ubicar el resultado en la comparación general. ¿Cuánto del ahorro del polifásico se recupera con data gating sobre la versión ingenua? ¿Queda alguna brecha? ¿De dónde viene?

Listing 1: Data gating por forzado a cero.

```
// Si tap_activo = 0 el operando queda quieto en cero
// y el multiplicador no conmuta
assign a_gated = tap_activo ? sample : '0;
assign prod = a_gated * coef;
```

Preguntas para el informe

- ¿Conviene forzar a cero o retener el último valor? Pensar en las transiciones al entrar y al salir del aislamiento.
- ¿Cuándo el data gating *aumenta* la potencia?
- ¿Por qué el polifásico sigue siendo mejor (si lo es) aun con data gating aplicado? Pensar en registros, reloj y área.

Ejercicio 5: Clock gating

Red de reloj y registros

Objetivo

Reducir la potencia de la red de reloj y de los registros que no necesitan actualizarse todos los ciclos. En el polifásico esto es natural: los subfiltros solo tienen que recalcular cuando llega una muestra nueva, es decir 1 de cada L ciclos.

- Referencia:** escribir los registros de los subfiltros con un multiplexor de realimentación explícito (el `enable` a mano como mux más flip-flop). Medir.

- (b) **Clock gating inferido:** reescribirlos con el patrón de *enable* que la herramienta reconoce (Listado 2) y volver a sintetizar. Revisar `report_clock_gating`: cuántos registros quedaron con celda ICG y cuántos no, y por qué.
- (c) Configurar el estilo de ICG con `set_clock_gating_style` y variar el ancho mínimo de banco al que se le aplica (2, 4 y 8 bits). Analizar el compromiso entre cantidad de celdas ICG y ahorro.
- (d) Aplicar clock gating también a `interp_zeros` y comparar cuánto gana cada arquitectura. ¿Por qué el polifásico tiene más para ganar?
- (e) **Variante incorrecta, para discutir:** implementar la habilitación con una compuerta AND sobre el reloj en el RTL. Sintetizar, correr GLS **con SDF** y mostrar el problema en las ondas.

Listing 2: Registro con enable apto para inferencia de clock gating.

```
// La herramienta reemplaza el mux de realimentacion por una celda ICG
always_ff @(posedge i_clk) begin
    if (i_rst)
        delay_line[k] <= '0;
    else if (i_valid) // enable -> candidato a clock gating
        delay_line[k] <= x_in;
end
```

Nota

Fusion Compiler infiere *clock gating* **automáticamente** cuando reconoce el patrón: no hace falta ningún flag en `compile_fusion`. En la clase lo vimos en el codificador BCH, donde la herramienta insertó una celda CKGTPLT a partir de `i_valid` sin que se lo pidiéramos.

Preguntas para el informe

- ¿Qué componente se reduce más, switching o internal? ¿Por qué?
- ¿Por qué la celda ICG está basada en un latch y no en una AND? ¿Qué se ve exactamente en las ondas de (e)?
- ¿Qué impacto tiene el clock gating sobre el leakage y el área?
- La celda ICG agrega retardo de inserción en el reloj. ¿Cómo se ve eso en la GLS post-síntesis?

Ejercicio 6: Leakage: multi- V_t *Leakage***Objetivo**

Reducir el consumo estático con celdas de distinta tensión umbral y entender el compromiso con la velocidad.

Sobre la mejor variante obtenida hasta acá:

- (a) Sintetizar tres versiones usando **un solo flavor** como `target_library`: solo hvt, solo rvt y solo lvt, al período de trabajo. Reportar área, WNS, P_{dyn} y P_{leak} . ¿Las tres cierran timing?
- (b) Sintetizar con los cuatro flavors simultáneamente (como hace el `syn.tcl` del template) y reportar la distribución de celdas por grupo de V_t con `report_threshold_voltage_group`.

- (c) Barrer el período (el de trabajo, su mitad y su doble) en la versión multi- V_t y graficar el porcentaje de celdas LVT/SLVT y el leakage contra el período.
- (d) Analizar el leakage en `tt0p8v25c` y en `ss0p72v125c`. ¿Cambia la conclusión sobre qué versión conviene?

Preguntas para el informe

- ¿En qué caminos quedaron las celdas de bajo V_t ? Relacionar con `report_timing`.
- ¿Por qué el multi- V_t casi no modifica la potencia dinámica?
- El polifásico tiene menos área que la versión ingenua. ¿Cómo se traduce eso en leakage?
- Para una aplicación que pasa el 95 % del tiempo inactiva, ¿qué ataca mejor el consumo total: clock gating o multi- V_t ?

Ejercicio 7: (Opcional) MAC compartido y simetría *Energía por muestra*

Objetivo

Explorar dos variantes más de arquitectura. Suma nota pero no es obligatorio.

- (a) **MAC compartido:** en lugar de L subfiltros en paralelo, implementar el polifásico con **un solo** multiplicador multiplexado en el tiempo, que recorre los coeficientes a la tasa alta. Comparar potencia, área y leakage contra la versión paralela. ¿Menos área implica menos potencia?
- (b) **Simetría de coeficientes:** si el prototipo cumple $h[k] = h[N-1-k]$, pre-sumar las muestras simétricas y usar la mitad de los multiplicadores. Cuidado: la simetría del prototipo no se traslada directamente a cada subfiltro polifásico. Analizar si se puede aplicar y en cuál de las dos arquitecturas conviene.

Preguntas para el informe

- ¿Cuándo conviene compartir hardware y cuándo paralelizar? Relacionar con el compromiso entre potencia dinámica, leakage y área.

7. Ayudas y recordatorios

Todo esto lo vimos en clase. Son las situaciones con las que se van a cruzar en el laboratorio: tenerlas presentes antes de empezar ahorra bastante tiempo.

Recordatorio

Si aparecen miles de violaciones con un límite redondo (1000 ps), es un placeholder de la librería. Los modelos traen *placeholders* de 1,0 ns en \$setup, \$hold y \$width, pensados para que el SDF los sobrescriba. Si el SDF no trae ese chequeo, queda el placeholder y todo lo viola.

- **Hold:** Fusion Compiler solo escribe los chequeos activos en el escenario. Hay que habilitar hold **antes** de write_sdf (ver Apéndice A.2), o el SDF sale sin arcos HOLD.
- **Width:** FC no escribe WIDTH en el SDF. Si el período es menor a 2 ns, el semiperíodo es menor al placeholder y **se viola por construcción**. Es esperable; usar make gls-ntc para las corridas de potencia.

Regla práctica: un límite redondo en una violación es sospechoso. Ir al SDF y verificar si ese chequeo está anotado.

Recordatorio

La GLS post-síntesis puede mostrar violaciones de setup aunque la STA cierre con slack positivo. La STA de síntesis usa reloj ideal: en el reporte aparece clock network delay (ideal) 0.00. El netlist, en cambio, tiene celdas ICG y buffers con retardo real, que la GLS sí ejecuta. Es esperable post-síntesis y lo resuelve el CTS en P&R, donde el reloj pasa a ser propagado. Por eso la GLS de firma es la post-P&R.

Recordatorio

Para que se genere el FSDB hay que compilar con -kdb. El método viejo (-P .../no-vas.tab .../pli.a) está **deprecado desde Verdi 2024.09** y falla en silencio: compila sin error y no escribe nada. Hay que compilar con -kdb y tener VERDI_HOME exportado. El template ya lo hace.

Recordatorio

Ajustar el skew del *clocking block* al período de trabajo. Si tiene output #1ns y el período es menor a 1 ns, los datos caen **después del flanco siguiente**. El skew de salida debe ser mucho menor que el período y coherente con el set_input_delay del SDC.

Recordatorio

Verificar la cobertura del SAIF antes de crearle al número. Si el SAIF no cubre todas las nets, PrimeTime PX inventa un *toggle rate* por defecto para las que faltan. **Siempre** correr report_switching_activity -list_not_annotated y verificar que la anotación sea prácticamente del 100 %. Además, revisar que el -strip_path apunte a la instancia correcta del DUT dentro del testbench.

Recordatorio

Comparar siempre a la misma frecuencia. La potencia dinámica es proporcional a f . Si el tb corre a una frecuencia y el SDC pide otra, el número no significa lo que parece. Fijar el período y usar el mismo en todas las variantes, o comparar E_{muestra} .

8. Entregables

8.1. Tabla 1: comparativa de arquitecturas (la principal)

Es el resultado central del TP. Las dos implementaciones, mismo estímulo, mismo período de trabajo, misma verificación:

Magnitud	Ingenua	Polifásica	Dif. [%]
Multiplicadores instanciados			
Multiplicaciones por muestra de salida			
$f_{\text{máx}}$ propio [MHz]			
WNS al período de trabajo [ns]			
Área [μm^2]			
Cantidad de registros			
P_{sw} [μW]			
P_{int} [μW]			
P_{leak} [μW]			
P_{tot} [μW]			
Potencia en multiplicadores [μW]			
Potencia de glitch [μW]			
Glitches totales (suma IG)			
E_{muestra} [pJ]			

8.2. Tabla 2: variantes con técnicas de reducción

Una fila por variante evaluada, todas en las condiciones comunes:

Variante	Área [μm^2]	WNS [ns]	P_{sw} [μW]	P_{int} [μW]	P_{leak} [μW]	P_{tot} [μW]	E_{muestra} [pJ]
interp_zeros (ref.)							
+ data gating (forzado)							
+ data gating (retención)							
+ clock gating							
interp_poly							
+ clock gating							
+ multi- V_t							
MAC compartido (opcional)							

8.3. Informe

- Descripción breve de cada modificación RTL (diff o fragmento relevante).
- Los gráficos pedidos en cada ejercicio y las respuestas a las preguntas.
- Evidencia de que cada variante pasa la verificación funcional.
- La captura de Verdi del Ejercicio 3(e).
- **Conclusión:** responder explícitamente cuánto cuesta multiplicar por cero, con los números de la Tabla 1 y el mecanismo físico detrás. Después, proponer el diseño final de menor E_{muestra} que cumpla timing y especificación, justificando qué técnica aportó más en cada componente.

8.4. Cronograma sugerido (4 semanas)

Semana	Ejercicios	Hito
1	1	Las dos arquitecturas escritas y verificadas; flujo andando; período fijado
2	2 y 3	Tabla 1 completa: la comparativa ingenua contra polifásica
3	4 y 5	Data gating y clock gating cuantificados
4	6, opcional y cierre	Tabla 2 completa e informe escrito

Nota

El hito de la semana 1 es el más importante: si el flujo no cierra, el resto no se puede medir. Si al final de la primera semana no tienen las dos arquitecturas verificadas y **make power** dando un número con anotación del 100 %, consultar.

8.5. Criterios de evaluación

Criterio	Peso
Flujo reproducible (scripts, Makefile, reportes ordenados)	15 %
Correctitud funcional de las variantes	15 %
Calidad y coherencia de las mediciones	25 %
Análisis y justificación física de los resultados	35 %
Presentación del informe	10 %

Nota

El peso mayor está en el **análisis**. Una medición que da distinto a lo esperado y está bien explicada vale más que una tabla llena de números sin discusión. Si algo no cierra, digan por qué creen que no cierra.

A. Plantillas de scripts

Todas estas plantillas están tomadas del flujo del curso y **corren en el laboratorio**. Ante cualquier diferencia de sintaxis, consultar **man <comando>** en la versión instalada.

A.1. Testbench: dump de FSDB

```
'ifndef FSDB_FILE
  'define FSDB_FILE "../sim/waves/interp_zeros_rtl.fsdb"
'endif

initial begin
  $fsdbDumpfile('FSDB_FILE);
  $fsdbDumpvars(0, tb_interp_zeros);
end

// Solo en el testbench de GLS: anotar los retardos del SDF
initial $sdf_annotate("../syn/outputs/interp_zeros.sdf", u_interp_zeros, , , "MAXIMUM"
);
```

```
# RTL (VERDI_HOME exportado, -kdb para el dumper de FSDB)
vcs -sverilog -full64 -timescale=1ns/1ps -debug_access+all -kdb \
  -F ../sim/tb/files.f -top tb_interp_zeros -o simv

# GLS con SDF
vcs -sverilog -full64 -timescale=1ns/1ps -debug_access+all -kdb \
  +neg_tchk -negdelay +sdfverbose $GLS_LIBS \
  -F ../gls/files_gls.f -top tb_interp_zeros_gls -o simv_gls

# GLS funcional (delay cero), para el Ejercicio 3
vcs ... +define+FUNCTIONAL $GLS_LIBS -F ../gls/files_gls.f -o simv_gls_func
```

```
# Actividad
fsdb2saif ../gls/waves/interp_zeros_gls.fsdb -o ../gls/waves/interp_zeros_gls.saif
```

A.2. Fusion Compiler: síntesis

```
set TOP $::env(TOP)
set LIB_BASE $::env(LIB_BASE)
set DB ${LIB_BASE}/db

# Corner slow: peor caso de setup
set SLOW_DBS [list \
    ${DB}/saed14hvt_base_ss0p72v125c.db ${DB}/saed14rvt_base_ss0p72v125c.db \
    ${DB}/saed14lvt_base_ss0p72v125c.db ${DB}/saed14slvt_base_ss0p72v125c.db \
    ${DB}/saed14hvt_cg_ss0p72v125c.db ${DB}/saed14rvt_cg_ss0p72v125c.db \
    ${DB}/saed14lvt_cg_ss0p72v125c.db ${DB}/saed14slvt_cg_ss0p72v125c.db ]
set link_library [concat "*" $SLOW_DBS]
set target_library $SLOW_DBS

# La lib de trabajo se borra siempre, asi el script es re-ejecutable
if {[get_libs -quiet ${TOP}.nlib] ne ""} { close_lib ${TOP}.nlib }
if {[file exists ${TOP}.nlib]} { file delete -force ${TOP}.nlib }
create_lib ${TOP}.nlib \
    -technology ${LIB_BASE}/physical/saed14nm_1p9m_mw.tf \
    -ref_libs $REF_NDMS

analyze -format sverilog ${PROJ_ROOT}/rtl/${TOP}.sv
elaborate ${TOP}
set_top_module ${TOP}

# Estilo de celda ICG (Ejercicio 5)
# set_clock_gating_style -sequential_cell latch
# Data gating automatico (Ejercicio 4)
# set_operand_isolation_style

compile_fusion

report_timing > timing.rpt
report_area > area.rpt
report_power > power.rpt
report_qor > qor.rpt
report_clock_gating > cg.rpt
report_threshold_voltage_group > vt.rpt

# CLAVE: habilitar hold antes de escribir el SDF, o sale sin arcos HOLD
current_scenario func_slow
set_scenario_status func_slow -setup true -hold true
write_sdf ${TOP}.sdf

write_verilog ${TOP}_netlist.v
write_sdc -output ${TOP}.sdc
save_lib
```

Nota

`compile_fusion` no lleva `-gate_clock`: el clock gating se infiere solo. Tampoco existe `report_power -groups` en Fusion Compiler; el reporte por grupos sale de PrimeTime PX.

A.3. PrimeTime PX: potencia

```

set TOP $::env(TOP)
set LIB_BASE $::env(LIB_BASE)
set DB ${LIB_BASE}/db

set link_path [list * \
  ${DB}/saed14hvt_base_ss0p72v125c.db ${DB}/saed14rvt_base_ss0p72v125c.db \
  ${DB}/saed14lvt_base_ss0p72v125c.db ${DB}/saed14slvt_base_ss0p72v125c.db \
  ${DB}/saed14hvt_cg_ss0p72v125c.db ${DB}/saed14rvt_cg_ss0p72v125c.db \
  ${DB}/saed14lvt_cg_ss0p72v125c.db ${DB}/saed14slvt_cg_ss0p72v125c.db ]

set power_enable_analysis true
set power_analysis_mode averaged

read_verilog ${PROJ_ROOT}/syn/outputs/${TOP}_netlist.v
current_design ${TOP}
link_design
read_sdc ${PROJ_ROOT}/syn/outputs/${TOP}.sdc

# El SAIF tiene la jerarquia del tb: hay que sacar el prefijo
read_saif ${PROJ_ROOT}/gls/waves/${TOP}_gls.saif \
  -strip_path tb_${TOP}_gls/u_${TOP}

update_power
report_power > power.rpt
report_power -hierarchy > power_hier.rpt
report_switching_activity -list_not_annotated > activity.rpt

```

A.4. Actividad supuesta (sin simulación)

Solo para el barrido exploratorio; **no** sirve para reportar resultados:

```

set_switching_activity -toggle_rate 0.25 -static_probability 0.5 \
  -base_clock clk [all_inputs]

```